D deploylab.

GUÍA COMPLETA · NIVEL INICIAL A INTERMEDIO

De localhost a producción.

Publica React + Vite en Vercel, conéctalo con Supabase y comprueba que funciona de forma segura.

01 GITHUB → 02 SUPABASE → 03 VERCEL

12 capítulos · código listo para copiar · imágenes · checklist

Incluye proyecto web interactivo y ejemplo ejecutable Vite + React.

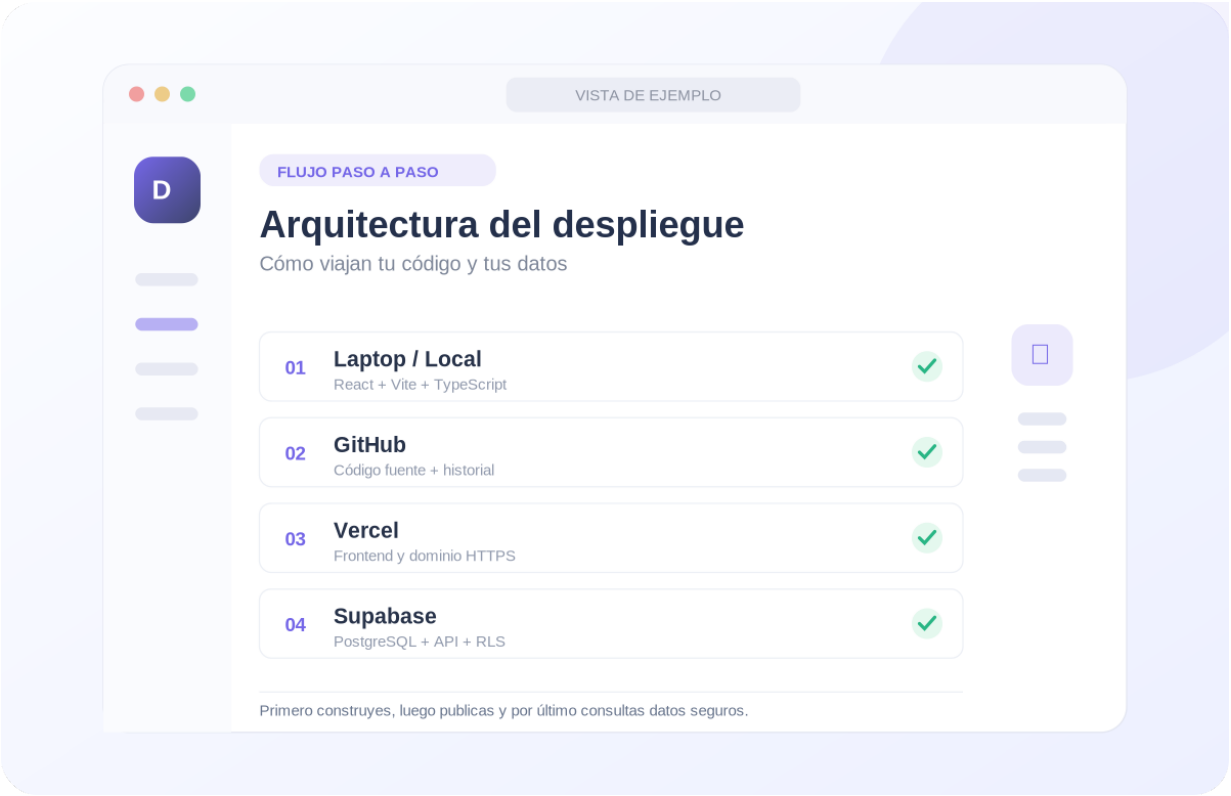
MATERIAL DE APOYO ACADÉMICO

Actualizado en noviembre de 2026 · Vercel + GitHub + Supabase

ANTES DE EMPEZAR

Tu mapa de despliegue

Esta guía se diseñó para que cualquier estudiante pueda desplegar su proyecto React + Vite + TypeScript. Lee los pasos, ejecuta los comandos, comprueba cada hito y vuelve al sitio interactivo para practicar.



¿Qué hace cada plataforma?

GitHub · Guarda tu código, commits y ramas. Conecta los cambios con Vercel.

Vercel · Recibe el repositorio, construye el frontend y publica una URL HTTPS.

Supabase · Aloja PostgreSQL, Auth, Storage y API. Necesita permisos mínimos y RLS.

GitHub Pages · Publica la guía educativa, que se exporta como HTML/CSS/JS estáticos.

CONSEJO · Para seguir la práctica necesitarás cuentas gratuitas o compatibles en GitHub, Vercel y Supabase, Node.js y Git. Los nombres de los menús pueden variar con actualizaciones.

CONTENIDOS

Tu recorrido, capítulo a capítulo

Puedes navegar este documento en el orden propuesto. Cada capítulo incluye una imagen ilustrativa, acciones, comprobaciones y fuentes oficiales.

01 Entiende qué se despliega y dónde · 5 min

02 Prepara tu proyecto local · 7 min

03 Publica el código en GitHub · 8 min

04 Crea un proyecto en Supabase · 9 min

05 Crea una tabla de prueba y configura RLS · 12 min

06 Conecta React con Supabase · 13 min

07 Importa tu repositorio en Vercel · 9 min

08 Configura Supabase dentro de Vercel · 10 min

09 Despliega y comprueba tu sitio real · 10 min

10 Si tu aplicación tiene inicio de sesión · 8 min

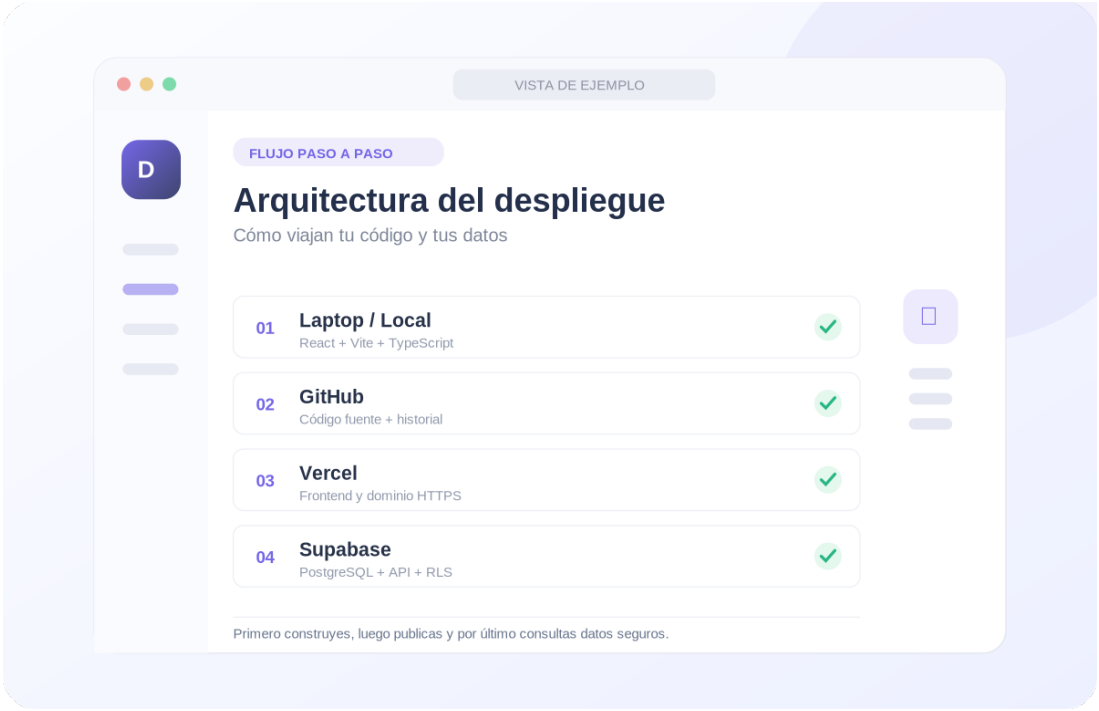
11 Diagnostica problemas frecuentes · 11 min

12 Publica esta guía en GitHub Pages · 10 min

CONSEJO · Los códigos de la versión web se pueden copiar con un clic. En el PDF se incluyen los ejemplos esenciales y todos los pasos.

Entiende qué se despliega y dónde

Antes de pulsar Deploy, separa frontend, hosting, repositorio y base de datos. No son la misma cosa.



PROCEDIMIENTO

01. Identifica las cuatro piezas

Tu computadora ejecuta la app durante el desarrollo. GitHub guarda el código y su historial. Vercel compila y sirve el frontend. Supabase ofrece PostgreSQL, autenticación, almacenamiento y una API.

- Frontend: interfaz construida con React + Vite que ve el usuario.
- GitHub: repositorio remoto, no es una base de datos.
- Vercel: crea un deployment y lo publica en un dominio HTTPS.
- Supabase: backend administrado, NO se despliega dentro de Vercel.

02. Recorre una petición real

El visitante abre https://mi-app.vercel.app, descarga los archivos del frontend, y su navegador utiliza el SDK de Supabase para consultar filas autorizadas por RLS.

- La clave publishable identifica la aplicación, no concede acceso ilimitado.
- Las políticas RLS deciden qué datos puede leer o modificar cada rol.
- Si el proyecto tiene un servidor propio, no se despliega automáticamente por publicar el frontend.

03. Elige tu escenario

Este recorrido es para aplicaciones React (preferentemente con Vite). La plataforma educativa que estás leyendo se creó con Next.js, pero NO necesitas Next.js para seguir los ejercicios.

ATENCIÓN · GitHub Pages sirve esta guía estática. No reemplaza a Vercel como plataforma donde se desplegará el frontend del ejercicio.

CHECKLIST DE COMPROBACIÓN

□ Sé distinguir frontend, repositorio, hosting y backend.

□ Puedo explicar por qué el navegador se comunica con Supabase.

ENLACES OFICIALES

↗ Vercel: Deploying Git Repositories

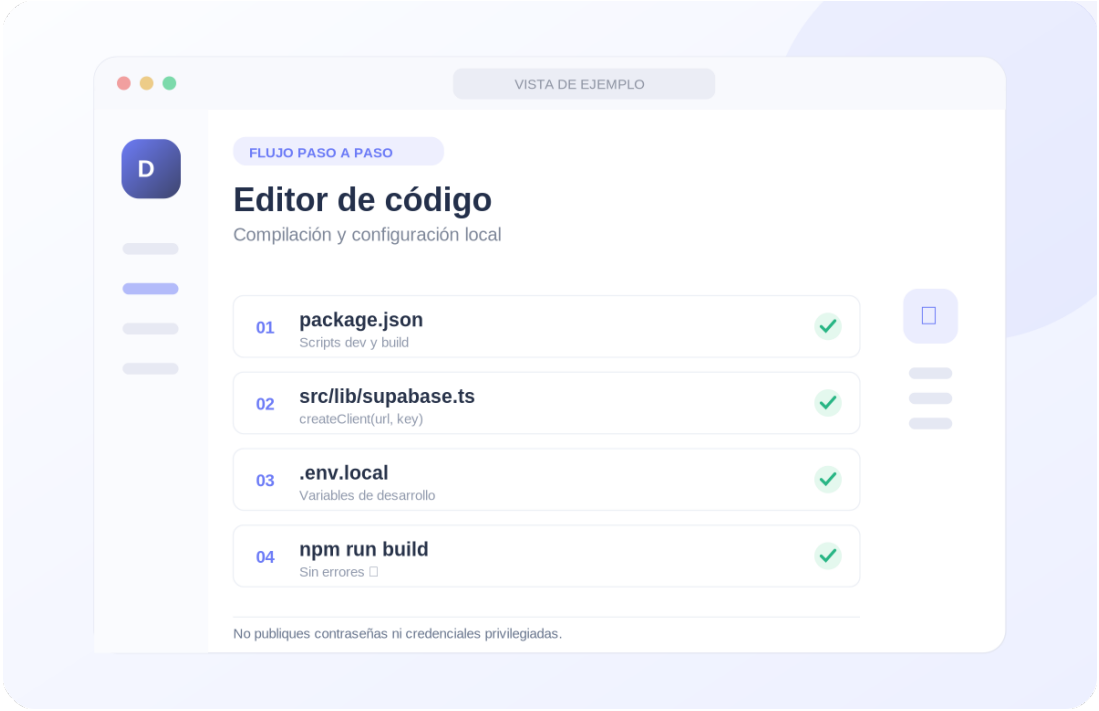
↗ Supabase: Quickstart React

---

02 · PREPARAR · CAPÍTULO 02/12

Prepara tu proyecto local

Una compilación local sin errores ahorra la mayoría de los problemas al desplegar.



PROCEDIMIENTO

01. Comprueba que tienes lo necesario

Necesitas Node.js compatible con tu proyecto React, Git, cuentas en GitHub, Vercel y Supabase. Identifica la carpeta que contiene package.json.

02. Si aún no tienes un proyecto React, créalo con Vite

Abre la terminal, ejecuta npm create vite@latest mi-proyecto -- --template react-ts, entra a la carpeta con cd mi-proyecto, ejecuta npm install y npm run dev. Si ya tienes tu proyecto React, omite este paso.

- react-ts genera React con TypeScript.
- npm run dev abre normalmente http://localhost:5173.
- Tu proyecto existente puede utilizar otra herramienta; verifica sus scripts en package.json.

03. Instala dependencias

Abre la terminal en la carpeta del proyecto (donde existe package.json) y ejecuta npm install. Revisa qué comando de construcción define scripts.build.

CONSEJO · Si tienes package-lock.json, usa npm ci en CI para instalaciones reproducibles.

04. Verifica que el frontend compila

Ejecuta npm run build. En React con Vite se creará dist/, que contendrá los archivos de producción que publicará Vercel.

- Revisa errores TypeScript.
- Comprueba rutas de importación y mayúsculas/minúsculas.
- Evita archivos pesados innecesarios o rutas absolutas a C:\Users\...

05. Protege las variables locales

Crea .env.local para desarrollo y asegúrate de que esté ignorado por Git. Nunca subas contraseñas, cadenas privadas de conexión ni claves secret/service_role.

ATENCIÓN · Las variables VITE_ se incorporan al JavaScript del navegador; nunca incluyas secretos. Solo URL y clave publishable de Supabase.

EJEMPLOS PARA COPIAR Y ADAPTAR

Crear React + Vite + TypeScript (si no tienes proyecto)

```
npm create vite@latest mi-proyecto -- --template react-ts
cd mi-proyecto
npm install
npm run dev
```

Verificar tu proyecto React existente

```
node --version
npm --version
git --version
npm install
npm run build
# Debe generarse dist/
```

.gitignore

```
node_modules/
.env
.env.local
.env.*.local
dist/
.next/
out/
```

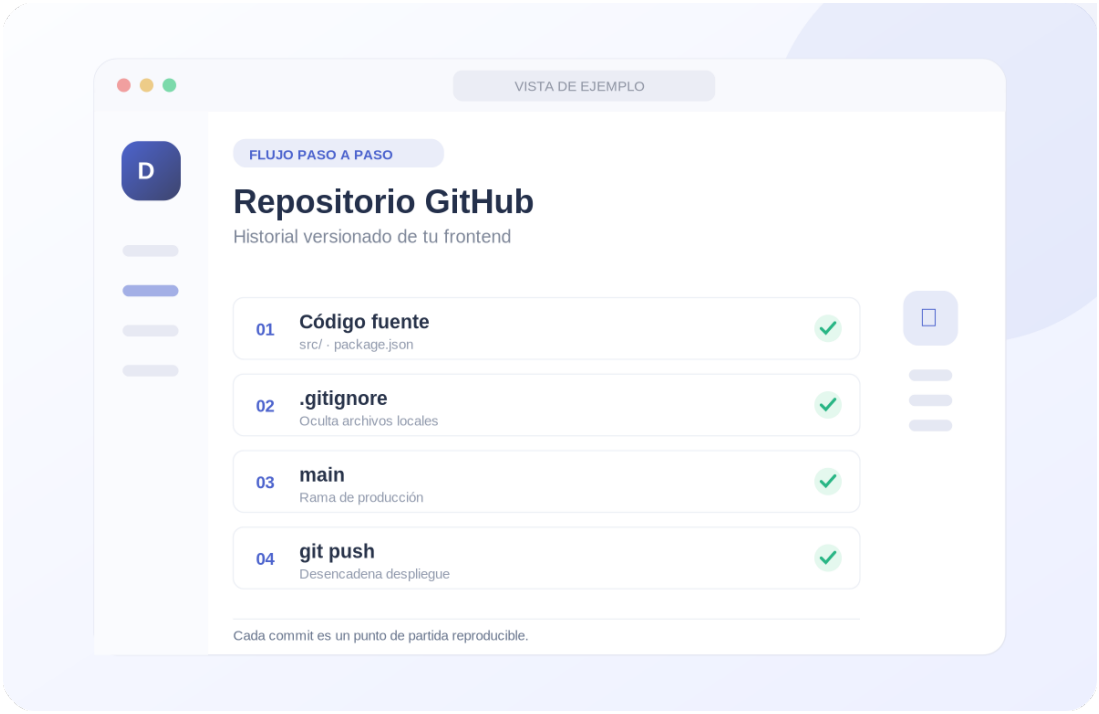
CHECKLIST DE COMPROBACIÓN

□ npm run build termina correctamente.

□ No hay secretos guardados en el repositorio.

Publica el código en GitHub

Vercel importará el repositorio y volverá a desplegar cuando actualices la rama de producción.



PROCEDIMIENTO

01. Crea un repositorio

Ve a github.com → New repository. Asigna un nombre sencillo, por ejemplo mi-primer-frontend. Puede ser público o privado. No subas .env.local.

02. Inicializa y registra tus archivos

Desde la terminal del proyecto usa git init si todavía no hay repositorio. Revisa git status y crea un commit.

- git add . incluye archivos no ignorados.
- git commit guarda el estado actual.
- git branch -M main define la rama de producción habitual.

03. Conecta el remoto y envía el código

Copia la dirección HTTPS de tu repositorio nuevo. Sustituye USUARIO y REPOSITORIO en los comandos. Si tu repo ya tiene historial, adapta el flujo y no reinicialices.

ATENCIÓN · Si GitHub rechaza el push por historial divergente, no utilices --force sin entender la situación.

04. Comprueba GitHub

Abre la URL del repositorio y confirma que ves package.json, src/ y el historial del commit. Tu archivo .env.local no debe aparecer.

EJEMPLOS PARA COPIAR Y ADAPTAR

Git / terminal

```
git init
git add .
git commit -m "Preparar frontend para despliegue"
git branch -M main
git remote add origin https://github.com/USUARIO/REPOSITORIO.git
git push -u origin main
```

CHECKLIST DE COMPROBACIÓN

□ El repositorio contiene el código actualizado.

□ El archivo .env.local NO aparece en GitHub.

ENLACES OFICIALES

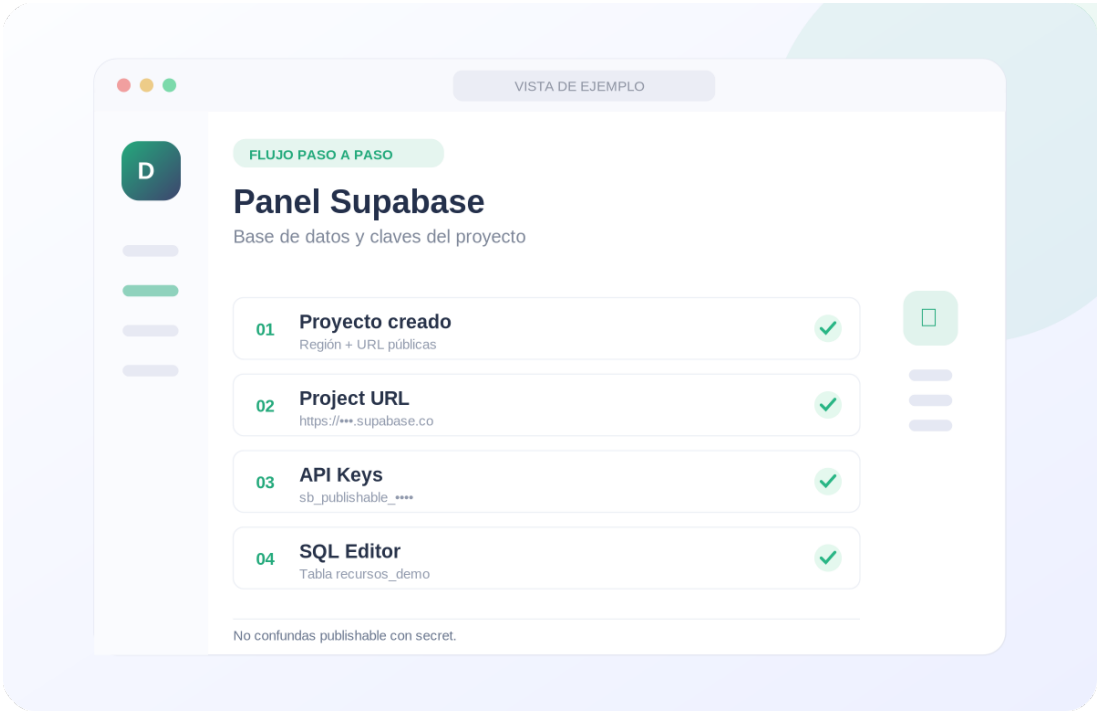
↗ GitHub: crear repositorio

---

04 · BACKEND · CAPÍTULO 04/12

Crea un proyecto en Supabase

Obtendrás un servidor PostgreSQL y las dos piezas de conexión públicas que necesita tu frontend.



PROCEDIMIENTO

01. Inicia sesión y crea un proyecto

Abre supabase.com/dashboard → New project. Selecciona tu organización y completa nombre, contraseña de la base de datos y región cercana a tus usuarios.

- Usa una contraseña fuerte y guárdala de forma segura.
- La región influye en la latencia.
- Espera que la creación concluya antes de configurar tablas.

02. Ubica la URL del proyecto

En el panel Connect o en Project Settings encontrarás la URL con formato https://xxxxxxxx.supabase.co. Copia únicamente la URL pública.

03. Localiza la clave publishable

En Connect / API Keys identifica la clave que comienza por sb_publishable_. Esa es la clave destinada a aplicaciones de navegador.

CONSEJO · Proyectos antiguos pueden mostrar una clave anon legacy. Para proyectos nuevos, utiliza preferentemente publishable.

04. Distingue claves públicas y secretas

publishable se puede integrar al frontend; secret (sb_secret_...) o service_role son exclusivamente para servidores confiables, nunca para un bundle frontend.

ATENCIÓN · La publishable key NO reemplaza el control de acceso. Debes configurar RLS y permisos SQL.

CHECKLIST DE COMPROBACIÓN

□ Tengo identificadas URL y publishable key.

□ No he copiado ninguna secret key al frontend.

ENLACES OFICIALES

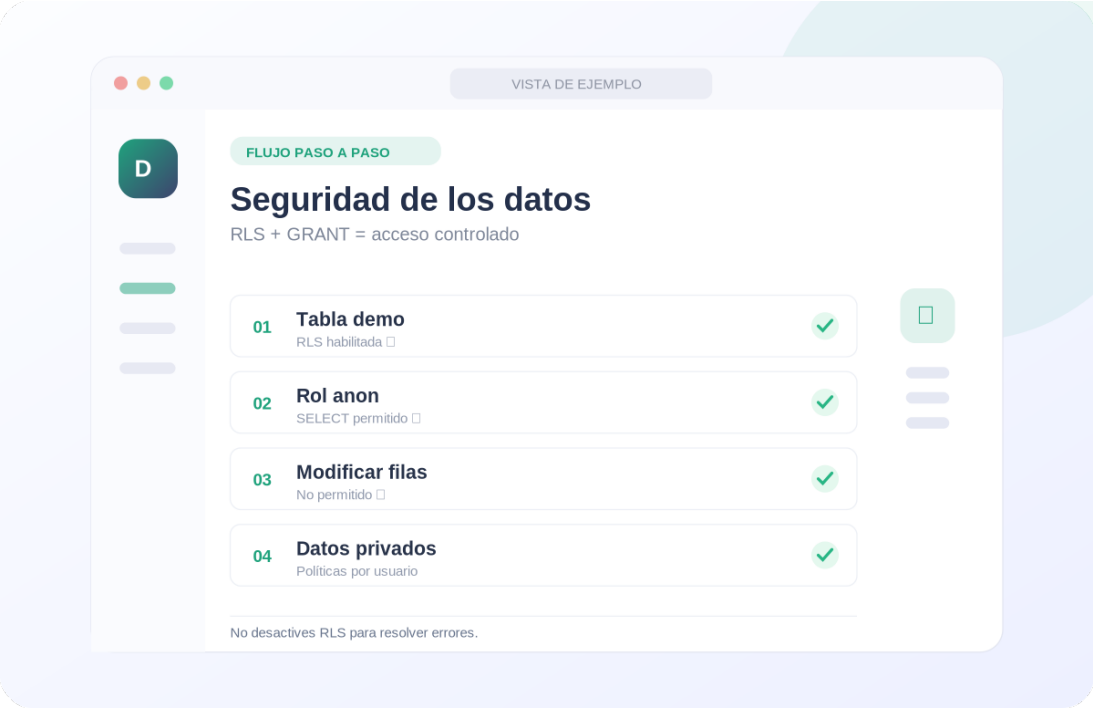
↗ Supabase: API Keys

---

05 · SEGURIDAD · CAPÍTULO 05/12

Crea una tabla de prueba y configura RLS

Prepara una consulta reproducible que sea segura: datos demo públicos, acceso de solo lectura y denegación de escritura anónima.



PROCEDIMIENTO

01. Abre SQL Editor

En Supabase selecciona SQL Editor → New query. Crearás una tabla de información de prueba, no una tabla de datos personales.

02. Ejecuta el SQL de ejemplo

Copia el bloque SQL completo y pulsa Run. Primero crea la tabla public.recursos_demo y dos registros; después activa RLS y revoca permisos generales a los roles de navegador.

- GRANT SELECT permite consultar la tabla.
- La política SELECT para anon habilita la lectura de esta tabla pública de demostración.
- INSERT, UPDATE y DELETE no se habilitan para el rol anon.

03. Confirma los datos

Ve a Table Editor → recursos_demo. Debes ver dos registros. Esta prueba usa deliberadamente datos públicos e inocuos.

04. Entiende la diferencia entre permiso y política

Un GRANT permite ejecutar una operación. Una política RLS restringe las filas disponibles para esa operación. En producción debes revisar ambas capas.

ATENCIÓN · No copies una política USING (true) a tablas con información privada. Esta política solo es adecuada para recursos deliberadamente públicos.

EJEMPLOS PARA COPIAR Y ADAPTAR

SQL seguro para demo pública

```
create table if not exists public.recursos_demo (
  id bigint generated always as identity primary key,
  titulo text not null,
  descripcion text not null
);

insert into public.recursos_demo (titulo, descripcion)
select 'Mi primer deploy', 'Frontend conectado a Supabase'
where not exists (select 1 from public.recursos_demo where titulo = 'Mi primer
   deploy');

insert into public.recursos_demo (titulo, descripcion)
select 'Aprendiendo RLS', 'Lectura pública de datos de prueba'
where not exists (select 1 from public.recursos_demo where titulo = 'Aprendiendo
   RLS');

alter table public.recursos_demo enable row level security;
revoke all on table public.recursos_demo from anon, authenticated;
grant select on table public.recursos_demo to anon;

drop policy if exists "Leer recursos demo publicos" on public.recursos_demo;
create policy "Leer recursos demo publicos"
on public.recursos_demo
for select to anon using (true);
```

Repetible en proyectos nuevos. La consulta WHERE NOT EXISTS evita duplicar los dos títulos de ejemplo.

CHECKLIST DE COMPROBACIÓN

□ RLS aparece habilitado.

□ La tabla solo contiene datos ficticios y públicos.

□ Los usuarios anónimos solo tienen SELECT.

ENLACES OFICIALES

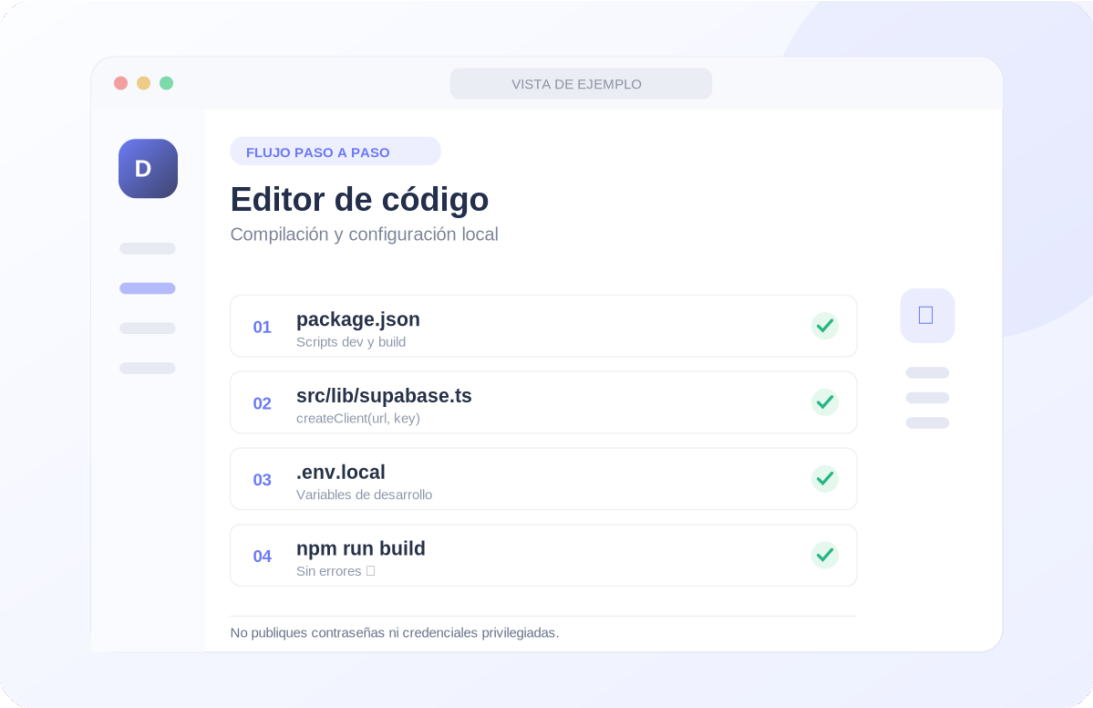
↗ Supabase: Row Level Security

---

06 · CÓDIGO · CAPÍTULO 06/12

Conecta React con Supabase

Instala la librería oficial, configura variables de entorno locales y haz tu primera lectura de datos.



PROCEDIMIENTO

01. Instala el SDK oficial

En la terminal del frontend ejecuta npm install @supabase/supabase-js. Este paquete crea un cliente para hablar con la API de Supabase.

02. Elige tu tipo de aplicación

Utiliza Vite + React: las dos variables públicas comienzan por VITE_. No uses variables de otros frameworks para este ejercicio.

- VITE_SUPABASE_URL: URL de tu proyecto Supabase.
- VITE_SUPABASE_PUBLISHABLE_KEY: clave que comienza por sb_publishable_.
- Las variables VITE_ se exponen al navegador; nunca deben contener secretos.

03. Crea el cliente

En src/lib/supabase.ts usa createClient(url, publishableKey). Añade una validación clara si faltan variables para detectar una configuración incorrecta.

04. Consulta recursos_demo

Dentro de un componente React carga los registros con supabase.from("recursos_demo").select(...). Muestra también estado de carga y error.

05. Prueba el resultado en local

Ejecuta npm run dev y abre la URL local. Debes ver dos títulos de ejemplo provenientes de la base de datos.

EJEMPLOS PARA COPIAR Y ADAPTAR

Instalación

```
npm install @supabase/supabase-js
```

Vite · .env.local

```
VITE_SUPABASE_URL=https://TU-PROYECTO.supabase.co
VITE_SUPABASE_PUBLISHABLE_KEY=sb_publishable_REEMPLAZAR
```

Vite · src/lib/supabase.ts

```
import { createClient } from '@supabase/supabase-js';

const url = import.meta.env.VITE_SUPABASE_URL;
const key = import.meta.env.VITE_SUPABASE_PUBLISHABLE_KEY;
if (!url || !key) throw new Error('Faltan variables de Supabase');
export const supabase = createClient(url, key);
```

React · src/Recursos.tsx

```
import { useEffect, useState } from 'react';
import { supabase } from './lib/supabase'; // Ajusta la ruta.

type Recurso = { id: number; titulo: string; descripcion: string };
export default function Recursos() {
  const [datos, setDatos] = useState<Recurso[]>([]);
  const [error, setError] = useState('');
  const [cargando, setCargando] = useState(true);
  useEffect(() => {
    supabase.from('recursos_demo').select('id,titulo,descripcion')
      .then(({ data, error }) => {
        if (error) setError(error.message);
        else setDatos(data ?? []);
        setCargando(false);
      });
  }, []);
  if (cargando) return <p>Cargando datos...</p>;
  if (error) return <p role="alert">Error: {error}</p>;
  return <ul>{datos.map(r => <li key={r.id}>{r.titulo}: {r.descripcion}</li>)}</ul>;
}
```

CHECKLIST DE COMPROBACIÓN

□ La instalación terminó sin errores.

□ Veo los dos registros en el navegador local.

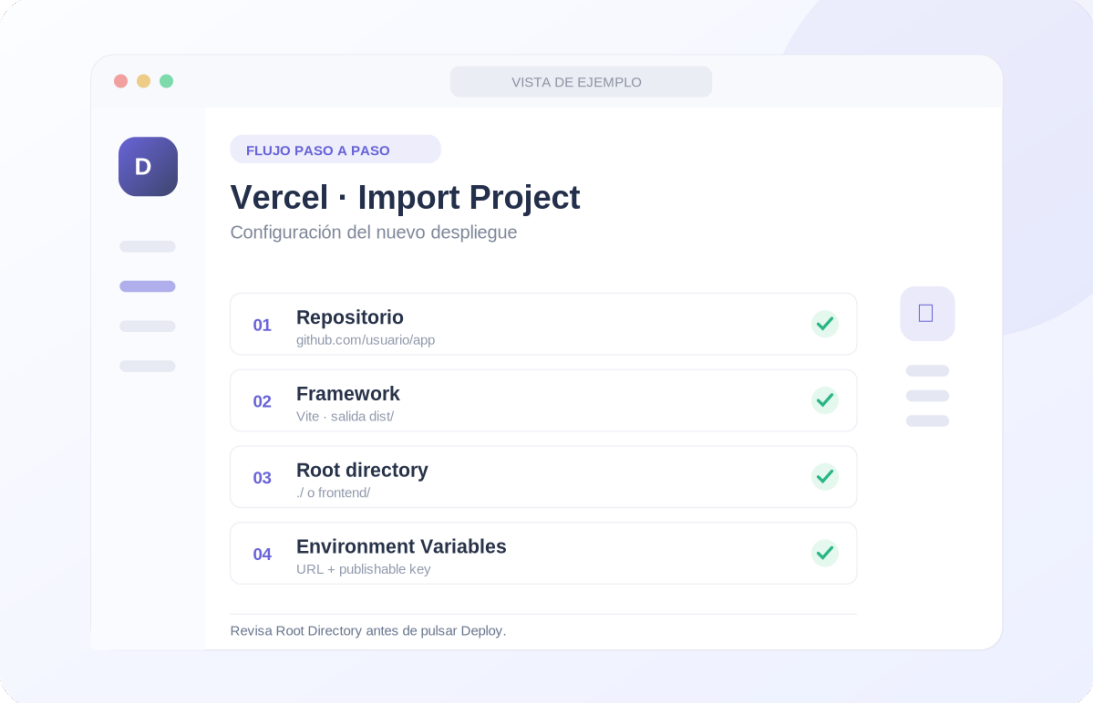
□ No aparece secret o service_role en el código.

ENLACES OFICIALES

↗ Supabase: React quickstart

Importa tu repositorio en Vercel

Conecta GitHub con Vercel y configura correctamente los parámetros del frontend.



PROCEDIMIENTO

01. Ingresa en Vercel

Abre vercel.com, inicia sesión y conecta tu cuenta GitHub si es la primera vez. Autoriza el repositorio que quieres desplegar.

02. Crea un proyecto nuevo

En el panel de Vercel selecciona Add New → Project o New Project. Busca tu repositorio y pulsa Import.

03. Revisa la carpeta raíz

Si package.json está en la raíz, usa el Root Directory predeterminado. Si está en frontend/ dentro de un monorepo, selecciona esa carpeta.

ATENCIÓN · Una Root Directory incorrecta suele provocar “No framework detected” o “package.json not found”.

04. Selecciona Vite como framework

En Vercel confirma Framework Preset: Vite, Build Command: npm run build y Output Directory: dist. Cuando Vercel autodetecte valores correctos, puedes mantenerlos.

- En Vite: Output Directory dist.
- Si package.json está en una subcarpeta, selecciona esa subcarpeta como Root Directory.
- Next.js es solo la tecnología de esta plataforma educativa, no de los proyectos de práctica.

05. Si utilizas React Router, prepara vercel.json

En una SPA con rutas como /dashboard o /perfil, Vercel puede devolver 404 al recargar una ruta interna. Coloca un archivo vercel.json en la raíz del frontend con una reescritura hacia /index.html. Si tu sitio no tiene rutas internas, puedes omitirlo.

CONSEJO · Prueba siempre una ruta interna pegando su URL directamente en una pestaña nueva.

06. No despliegues todavía sin revisar variables

Antes de pulsar Deploy, prepara las variables del siguiente capítulo. En algunos asistentes se pueden introducir durante la importación.

EJEMPLOS PARA COPIAR Y ADAPTAR

vercel.json · para React Router con Vite

```
{
  "$schema": "https://openapi.vercel.sh/vercel.json",
  "rewrites": [{ "source": "/(.*)", "destination": "/index.html" }]
}
```

CHECKLIST DE COMPROBACIÓN

□ Vercel puede leer mi repositorio.

□ Framework Preset = Vite y Output Directory = dist.

□ Si uso React Router, comprobé que una ruta interna carga al recargar.

ENLACES OFICIALES

↗ Vercel: Git deployment

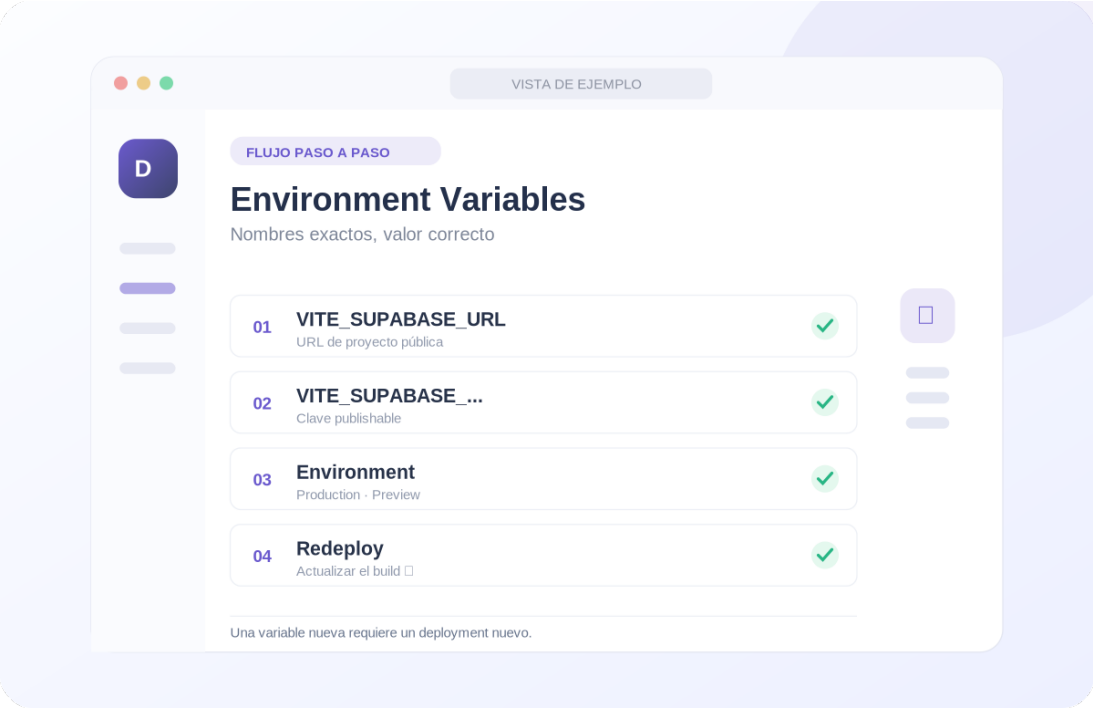
↗ Vercel: Vite / rutas SPA

---

08 · VARIABLES · CAPÍTULO 08/12

Configura Supabase dentro de Vercel

En producción, tu .env.local no viaja con Git: tienes que declarar las variables en Vercel.



PROCEDIMIENTO

01. Abre Environment Variables

En la creación del proyecto, expande Environment Variables. Si ya desplegaste, entra al proyecto → Settings → Environment Variables.

02. Agrega las dos variables exactas

Añade la URL y la publishable key del proyecto Supabase utilizando exactamente VITE_SUPABASE_URL y VITE_SUPABASE_PUBLISHABLE_KEY.

- Nombre: sin espacios, exactamente como el código.
- Valor: pega el dato del panel de Supabase.
- Entorno: selecciona Production; agrega Preview/Development si esas ramas deben funcionar.

03. Guarda y construye una nueva versión

Pulsa Save y posteriormente despliega o haz Redeploy si ya existía un deployment. Las variables nuevas no se incorporan retroactivamente a builds antiguos.

> CONSEJO · Una variable correctamente guardada no garantiza que el deployment anterior ya la utilice. Revisa fecha y estado del último despliegue.

04. Comprueba que no expusiste secretos

Únicamente URL + publishable key son aptas para el bundle web. Si pegaste una secret key, retírala del frontend y rótala según el procedimiento de Supabase.

EJEMPLOS PARA COPIAR Y ADAPTAR

Variables en Vercel (dos filas Name / Value)

```
VITE_SUPABASE_URL = https://TU-PROYECTO.supabase.co
VITE_SUPABASE_PUBLISHABLE_KEY = sb_publishable_REEMPLAZAR
```

CHECKLIST DE COMPROBACIÓN

□ Ambas variables existen en Production.

□ Volví a desplegar después de modificar variables.

ENLACES OFICIALES

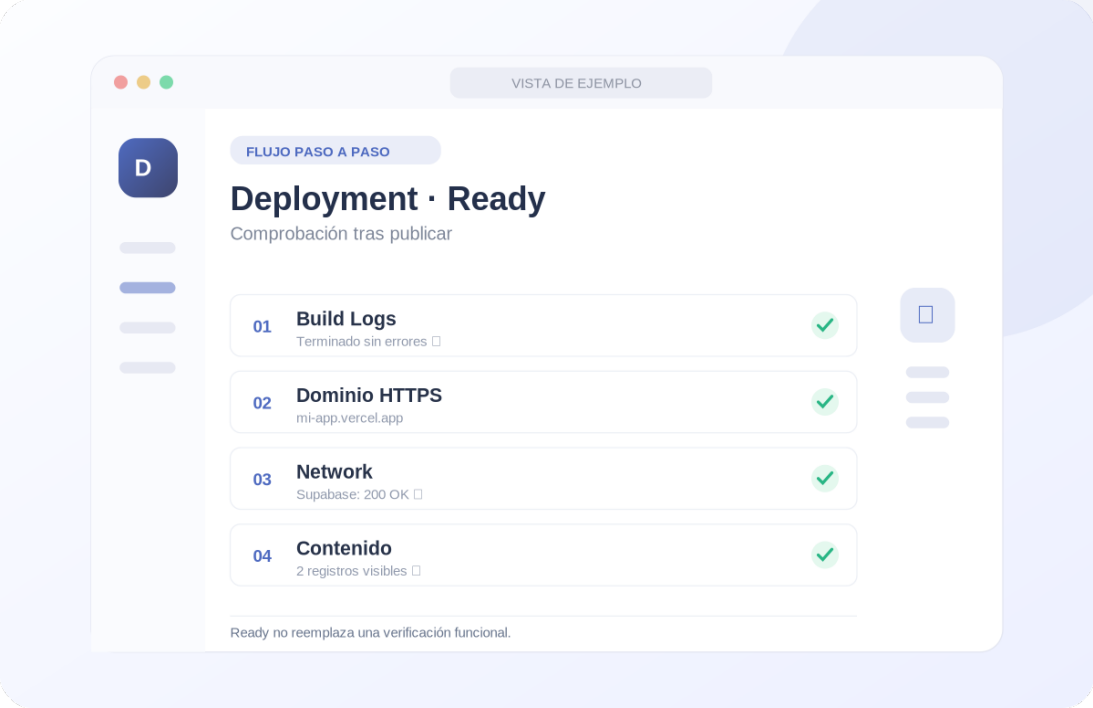
↗ Vercel: Variables de entorno

---

09 · PUBLICAR · CAPÍTULO 09/12

Despliega y comprueba tu sitio real

Un deployment exitoso no basta: hay que abrir la URL, comprobar los datos y revisar errores.



PROCEDIMIENTO

01. Pulsa Deploy

En Vercel haz clic en Deploy. Sigue el progreso de instalación de dependencias, build y publicación. Si falla, abre los Build Logs.

02. Abre el dominio de producción

Cuando veas Ready, abre la URL https://tu-proyecto.vercel.app. Prueba la aplicación desde una ventana privada o desde otro dispositivo.

- Comprueba que el sitio carga sin pantalla en blanco.
- Prueba rutas principales y recursos estáticos.
- Verifica que se muestran las filas de recursos_demo.

03. Inspecciona errores de red

Abre DevTools → Network y Console. Las consultas a Supabase deben responder correctamente. Si hay 401/403, revisa clave, permisos y RLS.

04. Verifica los próximos cambios

Haz un cambio visible pequeño, crea un commit y envíalo a main. Vercel hará un nuevo despliegue de producción si la integración y rama están configuradas correctamente.

05. Comparte el enlace

Copia el dominio HTTPS final. Ahora tu frontend está disponible en internet y la API Supabase responderá conforme a sus políticas.

EJEMPLOS PARA COPIAR Y ADAPTAR

Nueva versión por Git

```
git add .
git commit -m "Actualizar interfaz publicada"
git push origin main
```

CHECKLIST DE COMPROBACIÓN

□ El sitio abre por HTTPS.

□ La consulta muestra datos correctos.

□ Puedo publicar una actualización desde GitHub.

ENLACES OFICIALES

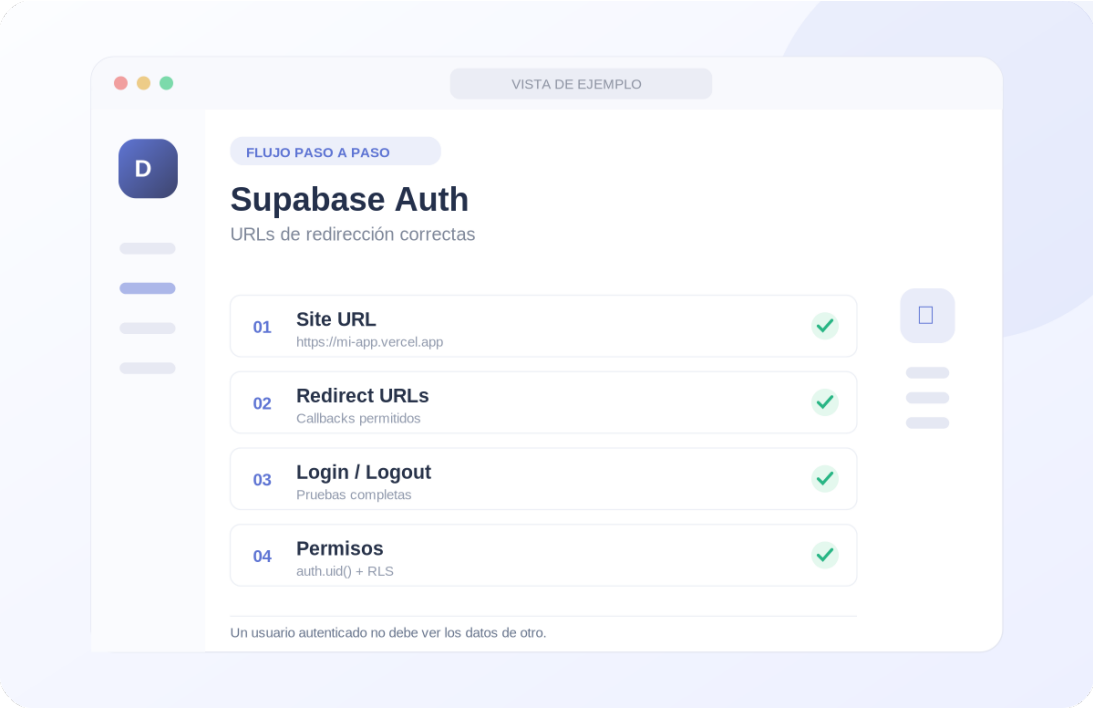
↗ Vercel: Deployments

---

10 · OPCIONAL · CAPÍTULO 10/12

Si tu aplicación tiene inicio de sesión

El correo de confirmación, enlaces mágicos y proveedores OAuth necesitan URLs correctas después del despliegue.



PROCEDIMIENTO

01. Define el Site URL de producción

En Supabase → Authentication → URL Configuration define Site URL con el dominio real HTTPS de Vercel.

02. Permite URLs de redirección necesarias

Añade http://localhost:5173/** para pruebas de React + Vite, además de las rutas de callback HTTPS que uses en el dominio de Vercel.

- Para una app con callback /auth/callback, autoriza la ruta exacta.
- Si usas OAuth, sigue también la configuración del proveedor.
- Para previews en Vercel consulta las reglas oficiales de comodines y limita su alcance.

03. Prueba los flujos completos

Regístrate, confirma correo, inicia y cierra sesión, recupera contraseña, recarga la página y verifica rutas protegidas.

04. Ajusta RLS para usuarios

Si manejas datos privados, usa políticas por usuario con auth.uid(), y evita usar una política pública para información sensible.

ATENCIÓN · Nunca consideres el hecho de ocultar un botón como autorización. La base de datos debe rechazar consultas no permitidas.

EJEMPLOS PARA COPIAR Y ADAPTAR

RLS de ejemplo privado

```
-- Ejemplo conceptual: adapta nombres y columnas a tu esquema.
-- La tabla debe tener una columna user_id uuid.
alter table public.mis_notas enable row level security;
revoke all on table public.mis_notas from anon, authenticated;
grant select on table public.mis_notas to authenticated;
create policy "Cada usuario ve sus notas"
on public.mis_notas for select to authenticated
using ((select auth.uid()) = user_id);
```

CHECKLIST DE COMPROBACIÓN

□ Site URL coincide con el dominio de producción.

□ Probé inicio, cierre de sesión y redirecciones.

□ Las tablas privadas tienen RLS restrictivo.

ENLACES OFICIALES

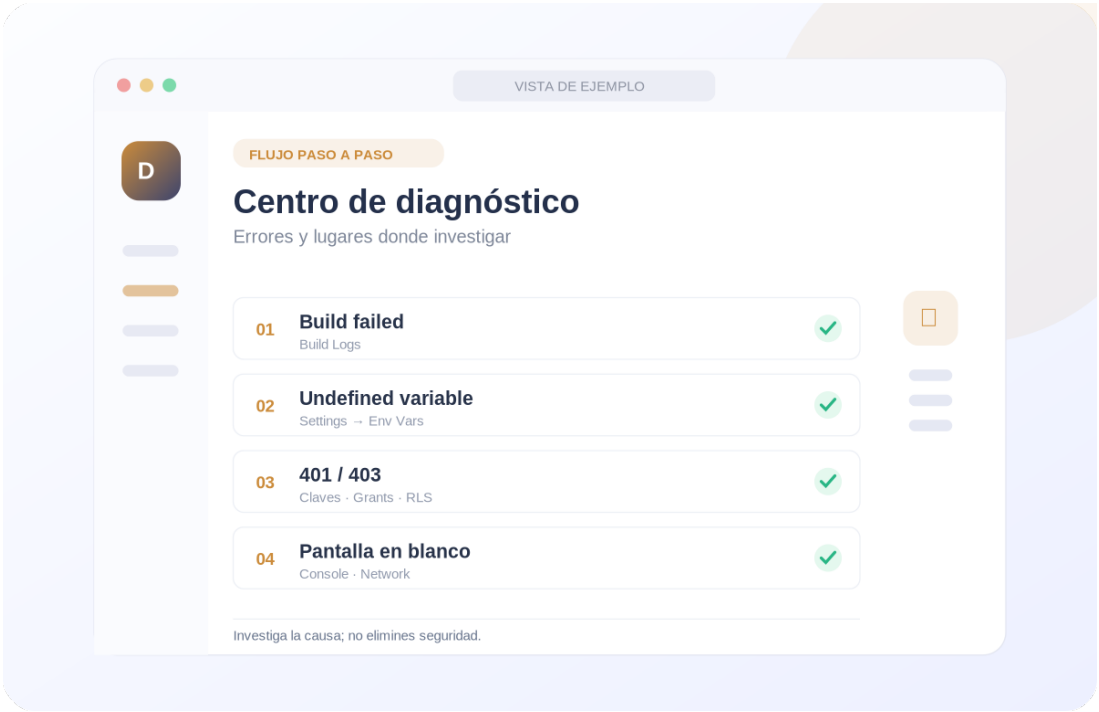
↗ Supabase: Redirect URLs

---

11 · SOLUCIONES · CAPÍTULO 11/12

Diagnostica problemas frecuentes

Aprende a investigar la causa antes de cambiar código a ciegas.



PROCEDIMIENTO

01. Si falla el build, abre los logs

En Vercel → Deployments → deployment fallido → Build Logs. Busca el primer error verdadero: TypeScript, dependencia ausente, ruta o framework.

02. Si aparece pantalla en blanco, revisa Console

Abre las herramientas del navegador. Errores de JavaScript o variables undefined suelen indicar configuración incorrecta, uso de window en el servidor o recursos mal enlazados.

03. Si la tabla está vacía, revisa permisos

Verifica datos existentes, nombre de la tabla, GRANT y política RLS, además de la respuesta en Network. Una consulta puede devolver cero filas por políticas, incluso sin excepción.

04. Si no se aplicó una variable, redeploy

En Settings → Environment Variables confirma nombres y entorno. Ejecuta un nuevo deployment para que se recompilen los valores del cliente.

05. Si falla el login, revisa URLs

Compara Site URL y Redirect URLs con el dominio publicado, protocolo HTTPS y ruta callback exacta.

CHECKLIST DE COMPROBACIÓN

□ Sé dónde encontrar Build Logs, Console y Network.

□ No desactivo RLS para “resolver” un error.

ENLACES OFICIALES

↗ Vercel: Managing Deployments

---

12 · BONUS · CAPÍTULO 12/12

Publica esta guía en GitHub Pages

Esta plataforma educativa ya está preparada para convertirse en archivos HTML/CSS/JS mediante la exportación estática de Next.js.



PROCEDIMIENTO

01. Descarga y descomprime la carpeta del proyecto

Instala dependencias con npm ci (si package-lock.json está disponible) o npm install para generar uno. Inicia con npm run dev y revisa el resultado.

02. Crea un repositorio para la guía

Usa, por ejemplo, deploylab-guia. Haz commit y push a main. El archivo .github/workflows/deploy-pages.yml viene incluido.

03. Activa GitHub Pages por Actions

En GitHub → Settings → Pages → Build and deployment → Source selecciona GitHub Actions. El workflow construye y publica out/.

04. Verifica la URL pública

Abre https://USUARIO.github.io/deploylab-guia/. El workflow establece REPO_NAME automáticamente para que Next.js genere las rutas correctas bajo el subdirectorio del repositorio.

- En un repositorio USUARIO.github.io, el sitio se publica en la raíz.
- Si cambias manualmente el nombre o usas un dominio propio, revisa basePath y la configuración del DNS.
- Esta guía no usa Server Actions ni rutas API: todo funciona como sitio estático.

05. Comparte el enlace y el PDF

Entrega a tus compañeros el enlace de la guía y el PDF. Sus avances personales se guardan solo en el almacenamiento local de su navegador.

EJEMPLOS PARA COPIAR Y ADAPTAR

Preparar y probar

```
npm install
npm run dev
npm run build
# La salida estática se genera en la carpeta out/
```

Publicar por Git

```
git init
git add .
git commit -m "Publicar guía interactiva"
git branch -M main
git remote add origin https://github.com/USUARIO/deploylab-guia.git
git push -u origin main
```

CHECKLIST DE COMPROBACIÓN

□ GitHub Actions está en verde.

□ El sitio funciona al entrar por GitHub Pages.

□ Las imágenes, animaciones y enlaces cargan en la URL pública.

ENLACES OFICIALES

↗ Next.js: static exports

↗ GitHub Pages: GitHub Actions

CIERRE · AUTOEVALUACIÓN

Repasa antes de publicar

Estas preguntas también están disponibles en el sitio web con corrección automática.

01. ¿Dónde se guarda el código fuente para que Vercel lo importe?

A) Supabase
B) GitHub
C) Chrome
D) Table Editor

02. ¿Qué clave Supabase puede utilizar el navegador?

A) sb_secret_...
B) service_role
C) sb_publishable_...
D) Contraseña PostgreSQL

03. ¿Qué cambia en Vercel después de editar una variable de entorno?

A) Nada, se actualiza el build viejo
B) Debes generar un nuevo deployment
C) Se elimina tu repositorio
D) Se desactiva RLS

04. ¿Qué mecanismo protege las filas de la base de datos?

A) Ocultar botones
B) Un archivo CSS
C) RLS y permisos SQL
D) Nombre secreto del repo

05. ¿Qué genera el build estático de esta guía para GitHub Pages?

A) Una instancia PostgreSQL
B) Archivos HTML/CSS/JS en out/
C) Un servidor Node persistente
D) Un contenedor Docker

06. ¿Qué significa que el deployment esté en estado Ready?

A) Toda la seguridad está verificada
B) El build y la publicación han terminado
C) Las políticas RLS se crean solas
D) El usuario ya ha probado los formularios

RESPUESTAS Y CONSULTAS

Solucionario y fuentes

01 B · GitHub aloja el repositorio; Vercel importa su código y crea deployments.

02 C · Publishable es la clave prevista para clientes públicos; RLS protege los datos.

03 B · Las variables se incorporan a los nuevos deployments, no retroactivamente.

04 C · Los controles de interfaz no sustituyen la autorización en PostgreSQL.

05 B · El output export de Next.js produce archivos estáticos que GitHub Pages puede servir.

06 B · Además de estar Ready, debes comprobar cargas, datos, logs, permisos y rutas.

DOCUMENTACIÓN OFICIAL

Vercel Git · https://vercel.com/docs/git

Vercel Vite y SPA · https://vercel.com/docs/frameworks/frontend/vite

Vercel Environment Variables · https://vercel.com/docs/environment-variables/managing-environment-variables

Supabase React · https://supabase.com/docs/guides/getting-started/quickstarts/reactjs

Supabase API Keys · https://supabase.com/docs/guides/getting-started/api-keys

Supabase RLS · https://supabase.com/docs/guides/database/postgres/row-level-security

Supabase Auth Redirect URLs · https://supabase.com/docs/guides/auth/redirect-urls

Next.js Static Export · https://nextjs.org/docs/app/guides/static-exports

GitHub Pages Actions · https://docs.github.com/en/pages/getting-started-with-github-pages/configuring-a-publishing-source-for-your-github-pages-site

CONSEJO · La guía incluye simulaciones educativas; no son capturas oficiales. No contiene credenciales reales. La versión web dispone de ejemplos copiables y una app Vite lista para personalizar.

¡Proyecto listo para compartir!

Descarga el ZIP del proyecto interactivo, súbelo a GitHub Pages y entrega el enlace. Quienes quieran practicar el despliegue real pueden utilizar la subcarpeta ejemplo-vite-supabase.